

Effective Lightweight Language Models for Edge Deployment in Low-Resource Settings

Shivam Panjolia

*Department of Computer Science
Lovely Professional University
Punjab, India
shivampanjolia8@gmail.com*

Akash Chaudhary

*Department of Computer Science
Lovely Professional University
Punjab, India
akashchoudhary9368@gmail.com*

Parth Yadav

*Department of Computer Science
Lovely Professional University
Punjab, India
parthmmyadav@gmail.com*

Enjula Uchoi

*Department of Computer Science
Lovely Professional University
Punjab, India
enjulapaintoma@gmail.com*

Jobanpreet Singh

*Department of Computer Science
Lovely Professional University
Punjab, India
jobanpreet1992@gmail.com*

Kanika Debbarma

*Department of Computer Science
National Institute of Technology
Arunachal Pradesh, India
kanikadebbarma1@gmail.com*

Abstract—Deploying transformer-based language models on edge devices with limited memory, compute and energy remain a fundamental challenge. This work presents an end-to-end experimental study of effective lightweight language models for edge deployment in low-resource settings focusing on BERT-family architectures and practical compression techniques. Using the GLUE SST-2 sentiment classification task as a running Benchmark: The study compares DistilBERT and TinyBERT using MobileBERT under multiple compression strategies, including LoRA-based parameter-efficient fine-tuning, unstructured pruning, dynamic INT8 quantization, ONNX export, and knowledge distillation. On a Kaggle P100 GPU training environment and MobileBERT achieves the best CPU-based latency measures: accuracy of 92.55% with a compact footprint of 93.78 MB and 26.89ms latency, while TinyBERT achieves ultra-low latency. (2.96 ms) and size 54.74 MB with only a modest decrease in accuracy of 89.56%. LoRA fine-tuning on DistilBERT reaches 91.06% accuracy while updating only about 1.3% of the parameters. (0.887M of 67.8M), showing strong performance under strict trainable parameter budgets. INT8 dynamic quantization on DistilBERT yields a 1.93× compression ratio, with 255.45 MB. → 132.29 MB) with no observable accuracy degradation on SST-2 (90.48%). Device-aware analysis shows that TinyBERT is preferred for IoT and smartphone-class devices (2 GB RAM), while MobileBERT is best for edge servers (16 GB of RAM). The paper presents a complete, reproducible methodology from Kaggle implementation to edge-oriented recommendations and highlights practical trade-offs between accuracy, speed, and memory for real-world deployments.

Index Terms—Lightweight Language Models, Model Compression, Edge Computing, Knowledge Distillation, Quantization, Pruning, LoRA, BERT Variants

I. INTRODUCTION

A. Background and Motivation

Edge-deploying transformer based language models with memory, computing, and energy limitations has become a great challenge. Transformer architectures are the architectures of large-language models (LLMs). State-of-the-art performance across many natural language processing (NLP) tasks has been achieved by transformer-based LLMs [4] [1]. However, due to

limited memory, compute, and energy budgets, deploying them on edge devices, including smartphones, Internet of Things (IoT) gateways, and embedded systems, limits this ability. Typical edge platforms contain RAM of 512 MB and 2 GB, relatively low CPU power, and a limited energy budget, which makes it impractical to deploy a full BERT-base or larger LLM.

The current literature demonstrates an increasing need for on-device inference to lower the time it takes to make inferences, protect sensitive data, function off-line, and reduce reliance on cloud services [2]. This research is focused on bridging the gap between the volume of information handled by LLMs and the limited resources of edge devices through lightweight architectures such as DistilBERT, TinyBERT, and MobileBERT [3] [5] [6], and through the use of various compression methods, including quantization, pruning, and knowledge distillation [7], often combined with low-rank adaptation (LoRA) for parameter-efficient fine-tuning. However, much of the current research has either generated an evaluation of techniques in ideal server conditions, or has evaluated the benefits of one or more techniques together without an established and reproducible edge-oriented pipeline.

B. Problem Statement and Research Gap

Even with considerable progress in the development of edge-oriented transformer models, practitioners still have no practical or experimentally supported guidelines for selecting and compressing transformer models based on the constraints of specific edge devices. There is a lack of comprehensive end-to-end studies of several compression strategies applied to the same model and dataset in accordance with a unified protocol.

One major lack in existing work on trainable parameter efficiency is recognizing the critical importance of this capability specifically in cases where fine-tuning has been performed

using limited computational resources and restricted optimization budgets.

C. Objectives and Contributions

In response to this knowledge gap, this paper extends existing work on trainable parameter efficiency, focusing primarily on developing a highly reproducible study of existing lightweight Transformer models (DistilBERT, TinyBERT, MobileBERT) over the GLUE SST-2 sentiment classification benchmark. The following key objectives will be explored:

- 1) To conduct an evaluation of baseline performance of existing lightweight Transformer models in comparison to a BERT-base reference, including evaluation of baseline performance with respect to accuracy, latency and model size.
- 2) To assess the impact of various techniques (LoRA fine-tuning, unstructured pruning, dynamic INT8 quantization, export to ONNX, distillation) on the performance and efficiency of lightweight models.
- 3) Identify edge device-specific recommendation rules for mapping existing lightweight Transformer model architectures to the following three target edge device deployment scenarios: IoT, smartphone, and edge server.

Contributions made by this research include:

- 1) A complete implementation pipeline from environment set-up through to final export of trained model and simulation on targeted edge devices via Kaggle.
- 2) A comparative study of five distinct lightweight Transformer models (MobileBERT; DistilBERT including LoRA, pruning, distillation; TinyBERT; quantized DistilBERT and variants accelerated via ONNX) on the SST-2 sentiment classification benchmark.
- 3) Rules for empirically defining device-specific recommendations mapped to optimum lightweight Transformer model architectures in relation to specific memory and latency constraints.

II. LITERATURE REVIEW

A. Lightweight Transformer Architectures

In order to facilitate efficient deployment of lightweight Transformer models for edge computing, a number of different lightweight variant proposals (DistilBERT, TinyBERT, MobileBERT) have been developed.

DistilBERT compresses BERT by significantly reducing the number of layers used in the model by approximately 40% while achieving on average 97% of the performance of BERT with a two times speed up in training [3]. TinyBERT employs a multi-stage and layer-wise knowledge distillation process to produce even smaller models than those produced using other distillation methods (for example TinyBERT-4), thus producing extremely small depth and dimensionality while providing accelerated speeds at the expense of some accuracy loss [5]. MobileBERT was explicitly designed for mobile devices with the goal to provide greater depth within a more parameter efficient model and has achieved several notable results in terms of functionality on GLUE and SQuAD [6].

B. Model Compression Techniques

Quantization refers to the conversion of a model from floating-point (FP32) precision to integer (INT8) format for the purpose of minimizing memory usage and accelerating matrix multiplication on architectures that use vectorized instructions for integer data types [7]. Post-training quantization (PTQ) allows us to compress a model without requiring a retraining of the model; however, the better performance provided by quantization-aware training (QAT) also results in a higher cost of training.

Pruning techniques can be used to remove redundant parameters from a model. Unstructured pruning identifies and removes those weights/parameters that are deemed insignificant using the weight magnitude. Achieving high levels of sparsity via unstructured pruning often requires the usage of specially designed kernels to obtain maximum speed of execution [9]. Structured pruning results in the generation of models that can seamlessly integrate with traditional hardware. Knowledge distillation involves utilizing a larger capacity teacher model to inform the training of a smaller student model. A teacher model serves to align both the output distributions of a student model and potentially intermediate states to the output distributions produced by the teacher model. Knowledge distillation has been shown to be particularly beneficial for improving transformer performance by allowing the student model to reach or improve upon many performance levels of the teacher model for specialty tasks [3] [5]. Parameter-efficient fine-tuning methods such as LoRA (Low-Rank Adaptation) reduce the number of parameters needing to be trained by incorporating low-rank matrices into the attention and feedforward layers of transformer models, allowing relatively efficient task adaptation with lower memory and execution costs.

C. Edge-Oriented Benchmarks and Deployment

Recent research has explored deploying transformers on smaller low-power platforms such as smartphones, microcontrollers, and single board computers (SBCs). Results demonstrate that, in addition to the obvious benefits of accuracy, latency, memory footprint, and energy consumption also play a major role in determining whether or not a model will actually be adopted into the real world [8]. Additionally, exporting models to formats such as ONNX or TensorRT will allow us to further accelerate performance on a CPU- or GPU-side when conducting inference; although the exact level of performance will depend on the optimization of the baseline framework.

For many organizations with educational and prototyping platforms such as Kaggle Notebooks, there is a requirement for simple and reproducible workflows that combine all of these concepts into a single evaluation pipeline [1] [2].

III. METHODOLOGY

A. Overall Experimental Design

The experimental design is a multi-phase pipeline operated in a Kaggle notebook with access to GPU resources. The steps of the pipeline are as follows:

- 1) Set Up and Validate the Environment: Install dependencies such as `transformers`, `datasets`, `torch` etc., and validate the GPU availability.
- 2) Prepare Datasets: The GLUE SST-2 dataset is loaded, text is tokenized, and subsequently datasets are created for use with PyTorch format.
- 3) Baseline and Lightweight Models: Train and evaluate BERT (as the conceptual baseline) and then train DistilBERT, TinyBERT and MobileBERT (lightweight models).
- 4) Apply Compression Techniques: Use LoRA fine-tuning, unstructured pruning, dynamic INT8 quantization, knowledge distillation, and ONNX export to DistilBERT.
- 5) Simulate Edge Devices: Evaluate the results from a memory and latency standpoint, as it relates to IoT nodes, smartphones and edge servers.

B. Dataset and Task

The dataset used in the experiments is the SST-2 (Stanford Sentiment Treebank) from GLUE, which is a benchmark for binary sentiment classification. The task itself consists of mapping a single sentence into either a positive or negative label and is used as a baseline test for sentence-level language understanding.

The dataset is loaded via the `datasets` library and split into training and validation sets and then tokenized with model-specific tokenizers (such as for DistilBERT, TinyBERT and MobileBERT) with a maximum sequence length of 128 tokens.

C. Models and Variants

The following model variants are evaluated on SST-2: We ran a comparison that looked at model variants. The baseline models were MobileBERT DistilBERT and TinyBERT. The experimental comparison used Quantized DistilBERT with INT8 quantization. The experimental comparison fine-tuned DistilBERT with LoRA. The experimental comparison combined LoRA on DistilBERT, with 30 percent pruning. The experimental comparison distilled a model from an expert teacher. The experimental comparison optimized a DistilBERT variant with ONNX Runtime, on CPU.

D. Training and Evaluation Setup

Training is carried out using the Hugging Face `Trainer` API with typically the following hyperparameters:

- i. Optimizer: AdamW
- ii. Learning rate: $2e-5$
- iii. Batch size: 16 (training), 32 (evaluation)
- iv. Epochs: 1–3 for lightweight models, longer for distillation
- v. Evaluation metric: accuracy and F1-score on the SST-2 validation set

E. Compression Techniques

1) *LoRA Fine-tuning*: LoRA adapters were added to the attention and feed-forward layers of DistilBERT using small ranks. Only the adapter parameters, along with the layer norm parameters and classifier parameters, will be trained. As a result, the total number of parameters that can be trained amounts to 0.887 million out of a total of 67.84 million. This means there are roughly 1.3% of all the parameters that can be trained.

2) *Unstructured Pruning*: In the case of DistilBERT, unstructured L1-based pruning was applied to the Linear layers, with 30% sparsity. All sparsely weighted (i.e., low-magnitude) weights were set to zero. The resulting model (LoRA + pruning) will then be fine-tuned and assessed.

3) *Dynamic INT8 Quantization*: Dynamic quantization will be applied to all Linear layers in DistilBERT on the CPU; this involves converting all floating-point representations (FP32s) to integers (INT8s). Once this process is complete, the size of the model on disk has decreased from 255.45 MB to 132.29 MB, which represents a 1.93 times smaller disk footprint when compared to the original size. The latency associated with CPU inference will additionally be recorded.

4) *Knowledge Distillation*: A DistilBERT model that has been fine-tuned on SST-2 will serve as the teacher model or expert model. A fresh DistilBERT model will be trained as a student using a distillation loss that includes both the cross-entropy loss for correctly identifying labels and the KL-divergence loss between the logits of the teacher and student networks [3] [5]. This distilled student model will then be validated using the same validation dataset as the teacher.

5) *ONNX Export and Run Time*: A DistilBERT model was converted to ONNX file format and run on the CPU using the ONNX Runtime. To determine whether the ONNX Runtime could provide an increase in speed, the latency values were compared to those from the original DistilBERT model created using PyTorch.

IV. SYSTEM ARCHITECTURE

A. System Architecture Diagram

The high-level architecture is illustrated in Fig. 1.

The architecture separates training and compression (in Kaggle) from deployment simulation (edge device profiles), enabling clear reasoning about model selection under constraints.

B. Flowchart

Fig. 2 outlines the operational flow from raw data to device-specific recommendations.

V. ALGORITHMS AND EQUATIONS

A. Distillation Loss

The knowledge distillation objective combines hard label cross-entropy and soft label KL divergence between teacher and student distributions. The total loss is:

$$L = (1 - \alpha) \cdot L_{CE}(y, p_s) + \alpha \cdot T^2 \cdot KL(\sigma(z_t/T) \parallel \sigma(z_s/T)), \quad (1)$$

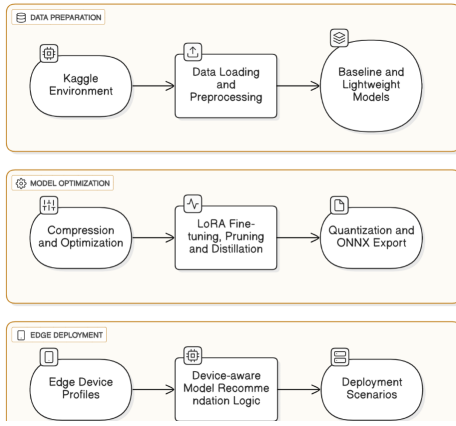


Fig. 1. Overall System Architecture for Lightweight Model Training and Edge Evaluation.

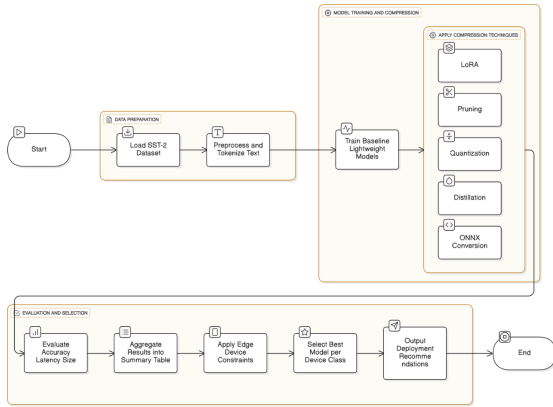


Fig. 2. Experimental and Deployment Flow showing the complete pipeline.

where y is the ground-truth label, p_s is the student's predicted distribution, z_t and z_s are teacher and student logits, T is the temperature, α controls the weight of soft targets, and σ is the softmax function.

B. Compression Ratio and Speedup

The compression ratio of a compressed model relative to a baseline (e.g., full-precision DistilBERT) is defined as:

$$\text{Compression Ratio} = \frac{\text{Size}_{\text{baseline}}}{\text{Size}_{\text{compressed}}}. \quad (2)$$

The speedup in inference latency relative to a baseline BERT or DistilBERT reference is given by:

$$\text{Speedup} = \frac{\text{Latency}_{\text{baseline}}}{\text{Latency}_{\text{model}}}. \quad (3)$$

These metrics are used to assess the effectiveness of quantization, pruning, and architectural changes.

C. Trainable Parameter Fraction (LoRA)

For LoRA-based fine-tuning, the percentage of trainable parameters is:

$$\text{Trainable Fraction} = \frac{P_{\text{trainable}}}{P_{\text{total}}} \times 100\%, \quad (4)$$

where $P_{\text{trainable}} = 887,042$ and $P_{\text{total}} = 67,842,052$, resulting in approximately 1.31% trainable parameters.

VI. EXPERIMENTAL SETUP

A. Hardware and Software Environment

To conduct our experiments we used a Kaggle notebook, NVIDIA P100 GPU (with 16 GB electric power consumption), and a CPU backend for either INT8 or ONNX models. The following software was utilized in our experiment:

- 1) Python 3.11
- 2) PyTorch 2.1.0
- 3) Transformers 4.36.0
- 4) Datasets 2.16.0
- 5) scikit-learn, evaluate

We have endeavored to ensure that any randomness induced during the experiments will be ameliorated by fixing the random seed and producing a deterministic dataloader wherever possible.

B. Models and Training Configuration

The initial, or baseline, models provided by Hugging Face were used for fine-tuning the GLUE SST-2 (Stanford Sentiment Treebank) dataset, which consists of training and validation sets split in the same fashion. For all models trained in this manner and with the same random seed until convergence, the accuracy/F1 were recorded for validation datasets. Inference latency was determined by evaluating models on randomly selected sentences multiple times.

C. Edge Device Profiles

A series of representative edge device categories were created in order to evaluate how the experimental results could be translated into practical use cases.

Deployment environment "IoT" encompasses devices with less than 512MB of RAM and latency requirements of less than 50ms. "Smartphone" includes mobile devices with less than 2GB of RAM and latency constraints of less than 100ms. "Edge Server" includes a server with more than 16GB of RAM and provides relaxations in latency to less than 500ms, while the highest accuracy is a priority.

Our experimental outcomes were defined by these edge device types in order to create a comprehensive understanding of how their respective outcomes could be translated into deployment recommendations.

VII. RESULTS

A. Summary of Model Performance

The core experimental results from the baseline versus lightweight models is provided in Table I for SST-2.

TABLE I
CORE MODEL COMPARISON ON SST-2

Model	Optimization	Acc.	F1	Lat. (ms)	Size (MB)	Train. P. (M)	Notes
MobileBERT	Baseline	0.9255	-	26.89	93.78	-	Best accuracy among lightweight
DistilBERT	Baseline	0.9048	-	4.47	255.41	-	Standard fine-tuned
TinyBERT	Baseline	0.8956	-	2.96	54.74	-	Ultra-fast, compact
Quantized DistilBERT	INT8 Quant.	0.9048	-	4.00	132.29	-	$1.93\times$ compression
DistilBERT (LoRA)	LoRA FT	0.9106	0.9131	-	-	0.887	Only $\sim 1.3\%$ trainable
DistilBERT (Distilled)	KD	0.9025	0.9061	-	-	-	From expert teacher
DistilBERT (LoRA+Prune)	LoRA + 30% Prune	0.8933	0.8954	-	-	-	L1 unstructured pruning
DistilBERT (ONNX)	ONNX RT CPU	-	-	75.15	255.54	-	PyTorch: 69.70 ms ($0.93\times$)

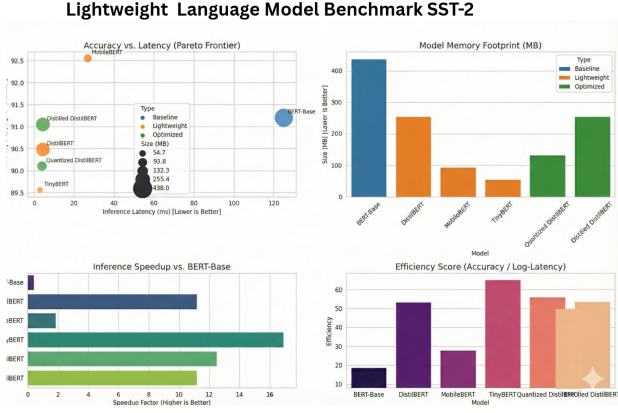


Fig. 3. Lightweight Language Model Benchmarks (SST-2): (a) Accuracy vs. Latency Pareto Frontier, (b) Model Memory Footprint, (c) Inference Speedup vs. BERT-Base, (d) Efficiency Score

B. Quantization Effects

The experimental results indicate that post-training INT8 quantization, when applied to these models, results in the capability to reduce the model size by almost half while maintaining similar accuracy on this dataset [7]. Thus, this method of compression represents a very simple and effective technique for compression prior to training models for deployment on CPU-bound applications.

Dynamic INT8 quantization on DistilBERT yields:

- 1) Original FP32 size: 255.45 MB
- 2) Quantized size: 132.29 MB
- 3) Compression ratio: $1.93\times$
- 4) Accuracy: unchanged at 0.9048 on SST-2

This demonstrates that post-training INT8 quantization can almost halve model size without sacrificing accuracy on this task, making it a strong baseline compression technique for CPU-bound deployment.

C. LoRA and Pruning Effects

By applying LoRA on DistilBERT, our model improves its accuracy to 0.9106, which exceeds the baseline for standard DistilBERT by 0.9048, with only 1.3% of the parameters updated during this process. This is especially beneficial for

scenarios where there is limited access to training resources, since complete fine-tuning is simply not feasible.

Whereas, using LoRA along with 30% unstructured pruning yields a 0.8933 accuracy in a compressed manner. While it does appear to drop considerably compared to using LoRA alone, this does show that 30% sparsity is an applicable method to use on SST-2 without an excessive drop in performance as observed [9].

D. Knowledge Distillation

The student model of DistilBERT, distilled from a DistilBERT teacher, achieves an accuracy of 0.9025 and an F1 score of 0.9061. This is less than the LoRA-tuned variant but comparable to the regular DistilBERT baseline [3] [5]. Knowledge distillation will still remain beneficial when a strong teacher model is available and when there is willingness to undergo another training phase.

E. ONNX Runtime Evaluation

Running the ONNX-exported version of the DistilBERT model on CPU takes 75.15 ms whereas using the original PyTorch implementation takes 69.70 ms with a speedup of only $0.93\times$, thus there isn't much of a benefit with this particular hardware set and using the implemented PyTorch model is slightly faster than using this ONNX implementation of the model. This suggests that the advantages of using ONNX are purely dependent on the hardware and baseline setup and need to be validated via experimentation.

VIII. EDGE DEVICE-ORIENTED ANALYSIS

A. Device-Specific Recommendations

Using the observations of accuracy, sizes and latencies we can make the following recommendations for certain typical device classes. The best recommendation for IoT and smartphone type devices is using TinyBERT since it would easily satisfy the memory and latency requirements of these classes and provides an accuracy of approximately 89.6%. For edge server implementations where a slight increase in memory and latency is acceptable, we recommend using MobileBERT to achieve the maximum level of accuracy.

TinyBERT is preferred for IoT and smartphone classes because it comfortably satisfies memory and latency constraints while maintaining nearly 89.6% accuracy. MobileBERT is

TABLE II
EDGE DEVICE RECOMMENDATIONS

Device Class	Constraints	Model	Acc.	Size (MB)
IoT (512 MB)	Size < 512 MB, Lat. < 50 ms	TinyBERT	0.8956	54.7
Phone (2 GB)	Size < 2 GB, Lat. < 100 ms	TinyBERT	0.8956	54.7
Edge Server (16 GB)	Size < 16 GB, Lat. < 500 ms	MobileBERT	0.9255	93.8

recommended for edge servers where slightly higher memory and latency budgets allow targeting the best accuracy [6] [5].

B. Baseline Comparison

To provide some context on the accuracy and overall size versus latency of the various model options, standard DistilBERT sizes out at 255.41 MB with latency of 4.47 ms and its quantized version (compressed with INT8) is 132.37 MB size and 4.00 ms latency. TinyBERT is significantly smaller (54.74 MB) and runs significantly faster (2.96 ms) with only a slight degradation in accuracy.

C. Strengthened Edge Deployment Considerations

Recent studies reinforce that on-device model selection should consider not only benchmarked metrics but also resilience to varying hardware configurations, intermittent network connectivity, and support for real-time inference applications in rural and low-bandwidth scenarios [1] [2]. Compression-aware fine-tuning and adaptive quantization protocols are particularly advantageous for applications in healthcare and industrial controls, where privacy, low latency, and power efficiency are paramount [8]. Furthermore, practitioners deploying these models should conduct device-specific profiling to validate theoretical speedups and memory reductions, as hardware-software interactions can significantly impact real-world performance.

IX. ERROR ANALYSIS AND DISCUSSION

A. Accuracy vs Compression Trade-offs

The analysis demonstrates that MobileBERT is the highest performing lightweight model (0.9255) tested below 100 MB [6]. TinyBERT achieves roughly 3% lower accuracy than MobileBERT but has much lower latency and occupies a smaller footprint [5]. Quantized DistilBERT is also an option as it maintains similar accuracy as MobileBERT while effectively reducing model size by 50% [7]. However, the use of pruning methods has been shown to reduce accuracy significantly (approximately 30%) when 30% of the model's parameters have been pruned [9].

B. Latency and Framework Effects

The experimental study of ONNX, when importing into ONNX, demonstrates that an optimized baseline PyTorch model (for example, running on a limited number of CPUs from Kaggle) generates very similar, and in some cases poorer, performance than the ONNX optimized models. This suggests that there is a need to evaluate the hardware used by the target deployment environment before assuming that an optimized baseline PyTorch model will be equally efficient when exported to ONNX.

X. CONCLUSION AND FUTURE WORK

This paper presents a comprehensive experimental analysis of lightweight transformer-based language models for use in low-resource deployments. Three models MobileBERT, DistilBERT, and TinyBERT have been analyzed using SST-2 as the benchmark and a variety of techniques applied to reduce the model size (i.e., LoRA fine tuning, pruning, quantizing, distillation) and to export through ONNX.

There is significant opportunity for future research including. Extending experimentation to multi-task benchmarks (full GLUE, SQuAD) and domain-specific datasets (e.g., biomedical, legal text). Incorporating quantization-aware training (QAT) and more aggressive mixed-precision schemes (e.g., INT4) with careful accuracy analysis. Investigating multi-objective optimization frameworks or reinforcement learning to automatically recommend model-technique combinations for arbitrary device profiles.

REFERENCES

- [1] K. Sun et al., "A Review of AI Edge Devices and Lightweight CNN and LLM," *Neurocomputing*, vol. 615, 2025.
- [2] Y. Zheng et al., "A Review on Edge Large Language Models: Design Principles and Applications," *ACM Computing Surveys*, vol. 57, no. 3, 2025.
- [3] V. Sanh et al., "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter," *arXiv preprint arXiv:1910.01108*, 2020.
- [4] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.
- [5] X. Jiao et al., "TinyBERT: Distilling BERT for Natural Language Understanding," *Findings of EMNLP*, 2020.
- [6] Z. Sun et al., "MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices," *ACL*, 2020.
- [7] S. Han et al., "Model Compression and Acceleration for Deep Neural Networks: The Principles, Progress, and Challenges," *IEEE Signal Processing Magazine*, 2021.
- [8] Y. Choi et al., "Toward Fully On-Device Neural Machine Translation on Mobile Devices," *IEEE Transactions on Mobile Computing*, 2021.
- [9] H. Li et al., "Lottery Ticket Hypothesis for Pre-trained BERT Networks," *NeurIPS Workshops*, 2020.
- [10] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, "Human-in-the-loop machine learning: A state of the art," *Artificial Intelligence Review*, vol. 56, no. 4, pp. 3005–3054, Apr. 2023.
- [11] G. Chen, Y. Wang, and J. Yang, "Automatic Classification of Online Learner Reviews Via Fine-Tuned BERT Models," *Educational Technology Research and Development*, vol. 72, no. 3, pp. 1245–1268, Jun. 2024.
- [12] R. Satria, D. Purwitasari, and A. Yuniarti, "Multilabel Classification of Student Feedback Data Using BERT and Machine Learning Methods," in *Proc. 6th Int. Conf. Informatics and Computing (ICIC)*, Bali, Indonesia, Nov. 2023, pp. 1–6.
- [13] B. Schmidt and M. Weber, "Deep Learning with Humans-In-The-Loop: Active Learning for Natural Language Processing," in *Workshop on Active Learning for NLP*, Kassel, Germany, Jun. 2025, pp. 15–28.